

Raijin Coupling Library Manual

Version 0.5

Raijin Developer Team

May 14, 2026

*“Lo, I am a herald of the lightning,
and a heavy drop out of the cloud:
the lightning, however, is the SUPERMAN.”*

— Friedrich Nietzsche,
Thus Spoke Zarathustra

Contents

1	Introduction to Coupled Calculations	6
2	Introduction	8
2.1	Overview of Raijin	8
2.2	Coupling Overview	8
2.2.1	Model requirements for coupling	8
2.2.2	Coupling pattern	9
3	Preparation	10
3.1	Grid index table	10
3.2	Remapping table	10
4	How to Use the API Subroutines	12
4.1	Overview of API usage	12
4.2	Initialization	12
4.2.1	Overview of initialization	12
4.2.2	Pre-initialization	13
4.2.3	Initialize Raijin	13
4.2.4	Get MPI information	14
4.2.5	Set grid index	14
4.2.6	Remapping table setting	15
4.2.7	Exchange data setting	15
4.2.8	Set initial time	16
4.3	Processing before time integration	16
4.3.1	Overview of preprocessing	16
4.3.2	Exchange of miscellaneous information	17
4.3.3	Exchange of initial data	17
4.3.4	Set initial time	17
4.3.5	Put initial values for the first time step	17
4.4	Time integration	18
4.4.1	Overview of time integration	18
4.4.2	Setting the current time and ΔT	18
4.4.3	Getting data	19
4.4.4	Putting data	19
4.5	Finalization	19
4.6	Other important routines	19
4.6.1	Error handling routine	19
4.6.2	Component query routines	19

4.6.3	Execution status routine	20
4.6.4	MPI information routine	20
4.6.5	Log output routine	20
4.6.6	Calendar routines	20
4.6.7	User-defined interpolation code setting routine	20
5	Implementation of Coupling Code	21
5.1	Overview of implementation	21
5.2	Example: toy model	21
6	Implementation of Interpolation Code	23
6.1	Overview	23
6.2	Implementing a user-defined interpolation subroutine	23
6.3	Registering a user-defined interpolation subroutine	24
6.4	Example of registering a user-defined interpolation subroutine	24
7	API Reference	26
7.1	Raijin API subroutines	26
7.1.1	Initialization API	26
7.1.2	Time integration API	28
7.1.3	Finalization API	28
7.1.4	Utility API	29

Preface

This document is the manual for the coupler Raijin. The development of Jcup [1,2], the predecessor of Raijin, began in 2007 and continues to be improved as of 2026.

During this period, Jcup has been widely adopted as a foundational coupling library primarily for weather and climate models, including the coupling of the atmospheric model NICAM [3,4] with the ocean model COCO [5], the coupling with AI [6], and the coupling within ILS (Integrated Land Simulator) [7].

However, as its range of applications has expanded, performance issues have become apparent — specifically, the large overhead associated with data exchange. This overhead is primarily caused by three factors: (1) the complexity of internal processing related to data exchange within Jcup, (2) the MPI function usage and implementation for data transmission and reception, and (3) the interpolation calculation.

To overcome these weaknesses, a new coupler, Raijin, has been developed. Raijin means the Japanese god of thunder. The name represents fast data exchange, low overhead, and a design focused on coupling performance. Raijin has three major features: (1) support for a sufficient set of coupling patterns, (2) adoption of asynchronous communication, and (3) realization of send-side interpolation. These features make it possible to perform coupling calculations with stringent performance requirements, such as coupling a high-resolution atmospheric model with ILS.

It is our hope that this manual will help Raijin to be widely used in a diverse range of coupled calculations.

- [1] Takashi Arakawa, Hiromasa Yoshimura, Fuyuki Saito, and Koji Ogochi. Data exchange algorithm and software design of kakushin coupler jcup. *Procedia Computer Science*, 4:1516–1525, 2011. doi:10.1016/j.procs.2011.04.164.
- [2] T Arakawa, T Inoue, H Yashiro, and M Satoh. Coupling library jcup3: its philosophy and application. *Progress in Earth and Planetary Science*, 7(1), 2020. doi:10.1186/s40645-019-0320-z.
- [3] Masaki Satoh, Hirofumi Tomita, Hisashi Yashiro, Hiroaki Miura, Chihiro Kodama, Tatsuya Seiki, Akira Noda, Yohei Yamada, Daisuke Goto, Masahiro Sawada, Takemasa Miyoshi, Yosuke Niwa, Masayuki Hara, Tomoki Ohno, Shin-ichi Iga, Takashi Arakawa, Takahiro Inoue, and Hiroyasu Kubokawa. The non-hydrostatic icosahedral atmospheric model: description and development. *Prog Earth Planet Sci (Web)*, 1, 2014. doi:10.1016/j.jcp.2007.02.006.
- [4] Masaki Satoh, Bjorn Stevens, Falko Jutdt, Marat Khairoutdinov, Shian-Jian Lin, William M. Putman, and Peter Duben. Global cloud-resolving models and their applications for climate simulations: Nicam and beyond. *Current Climate Change Reports*, 5(3):172–184, 2019. doi:0.1007/s40641-019-00131-0.
- [5] T. Miyakawa, H. Yashiro, T. Suzuki, H. Tatebe, and M. Satoh. A madden-julian oscillation event remotely accelerates ocean upwelling to abruptly terminate the

- 1997/1998 super el niño. *Geophysical Research Letters*, 44(18):9489–9495, 2017. URL: <https://agupubs.onlinelibrary.wiley.com/doi/abs/10.1002/2017GL074683>, arXiv:<https://agupubs.onlinelibrary.wiley.com/doi/pdf/10.1002/2017GL074683>, doi:10.1002/2017GL074683.
- [6] Takashi Arakawa, Hisashi Yashiro, and Kengo Nakajima. Development of a coupler h3-open-util/mp. *International Conference on High Performance Computing in Asia-Pacific Region*, 2022. URL: <https://dl.acm.org/doi/pdf/10.1145/3492805.3492809>, doi:10.1145/3492805.3492809.
- [7] T Nitta, T Arakawa, M Hatono, A Takeshima, and K Yoshimura. Development of integrated land simulator. *Progress in Earth and Planetary Science*, 7(68), 2020. doi:10.1186/s40645-020-00383-7.

Chapter 1

Introduction to Coupled Calculations

This chapter provides an overview of coupled calculations. Readers already familiar with coupled calculations may skip this chapter.

First, the physical aspects of coupled calculations are described. Weather and climate result from the interaction of multiple physical components, such as the atmosphere and ocean. For example, the atmosphere transfers momentum to the ocean surface, while the ocean surface transfers sensible and latent heat to the lower atmosphere (Figure 1.1). A simulation model that represents these phenomena must therefore exchange the corresponding physical quantities between components. In such a simulation, each component model uses its own grid system and time step suited to the physical phenomena it represents. To exchange data between models with different grid systems and time steps, appropriate time management and grid remapping are required.

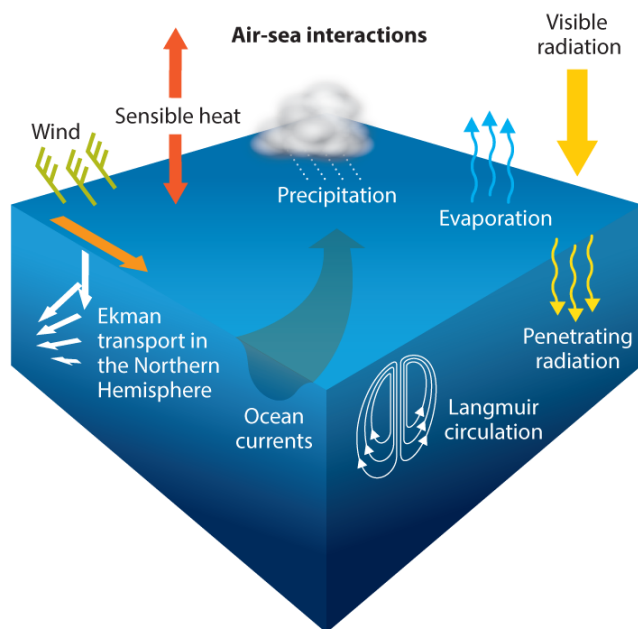


Figure 1.1: Schematic of atmosphere–ocean interaction (from Encyclopedia of the Environment, <https://www.encyclopedie-environnement.org/en/air-en/biosphere-hydrosphere-and-cryosphere-models/>).

Next, the computational aspects of coupled calculations are described. Modern high-performance computers operate multiple processing units in parallel. When running a simulation model on such a computer, the model must be designed to exploit the computer's parallel architecture. This is generally achieved through domain decomposition, in which the model grid is divided among processing units. In a coupled calculation, multiple models run in parallel. Therefore, data exchange must be performed between the appropriate processes of each model, taking into account their respective domain decompositions.

Figure 1.2 shows an example of such a data exchange. In this example, model A has an 8×8 grid divided into 4×4 regions, and model B has a 12×12 grid divided into 3×3 regions. Each region is numbered as shown in the figure. As indicated by the light green arrow, region 5 of model A exchanges data with regions 0, 1, 3, and 4 of model B. Conversely, as indicated by the dark green arrows, region 4 of model B exchanges data with regions 6, 7, 10, and 11 of model A.

Implementing such complex data exchange individually for each model is impractical. Instead, dedicated software called a *coupler* (or *coupling library*) is generally used. As described above, a coupler performs three main tasks:

1. Control of data exchange timing
2. Grid remapping
3. Data exchange between appropriate processes

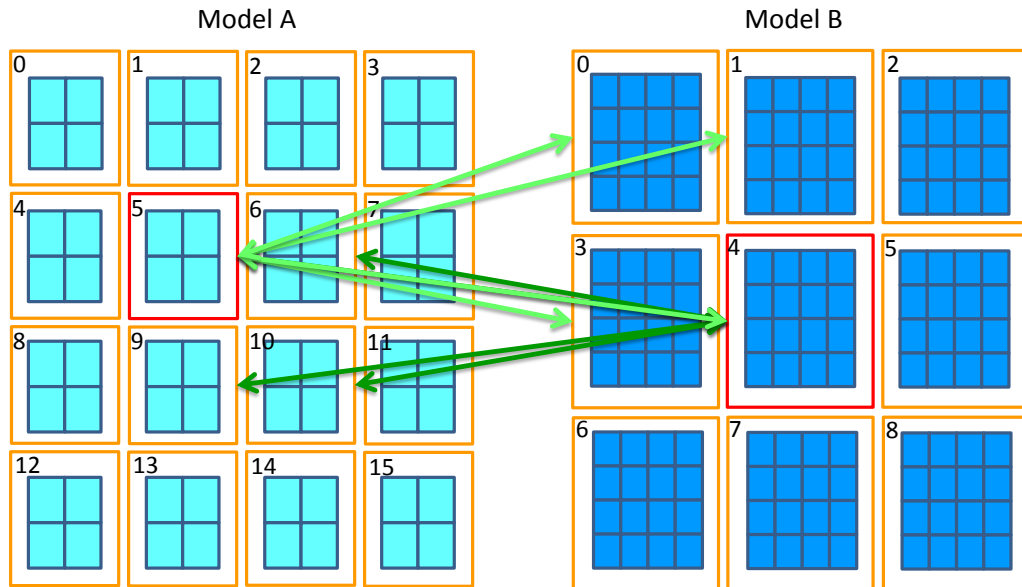


Figure 1.2: Schematic of data exchange between domain-decomposed models.

Chapter 2

Introduction

2.1 Overview of Raijin

Raijin is a lightweight coupling library derived from the coupling library Jcup. While Jcup is a general-purpose coupler that supports a wide range of coupling configurations, Raijin is designed to be simpler and faster, focusing on the core functionality needed for typical coupled simulations.

Raijin requires all coupling parameters to be specified through API subroutines. To use Raijin in a model component, include `rjn_interface.h` from C or C++ source code and link the Raijin library. Fortran programs can use the compatibility module `jlt_interface`, which forwards the original `jlt_*` API names to the Raijin implementation.

The main features of Raijin are as follows:

- Two or more components can be coupled
- Components can be coupled in series or in parallel
- Each component can be parallelized through domain decomposition
- Any grid system can be used, provided that each grid point has a unique index
- A single component can have multiple grid systems
- The data exchange interval can be set individually for each variable
- The number of exchangeable variables is unlimited
- All coupling parameters are configured through API subroutines, without requiring a configuration file

2.2 Coupling Overview

2.2.1 Model requirements for coupling

Each component model to be coupled must be either single-process (non-parallel) or parallelized by domain decomposition. The domain decomposition pattern must remain fixed throughout the simulation, and the grid point indices assigned to each subdomain must not change. Grid point indices must be unique across the entire model grid but do not need to be consecutive.

The data exchange interval for each variable is fixed and must be an integer multiple of the model time step. The model time step outside of data exchange times may vary.

2.2.2 Coupling pattern

Raijin supports coupling two or more component models in series or in parallel. Here, *serial coupling* refers to a pattern in which two components exchange data within the same time step, and *parallel coupling* refers to a pattern in which each component sends data from the current time step and receives data from the previous time step of the other component.

Figure 2.1 illustrates both patterns.

In the serial coupling example (left), model A passes its results to model B within the same time step, and model B then performs its calculation using those results. Because each component waits for the other to finish, the total execution time is approximately the sum of the individual execution times.

In the parallel coupling example (right), each component sends its results to the other component at each time step and receives results from the other component at the next time step. In this case, the total execution time is approximately that of the slowest component.

The gray squares before T_0 represent the initial value exchange.

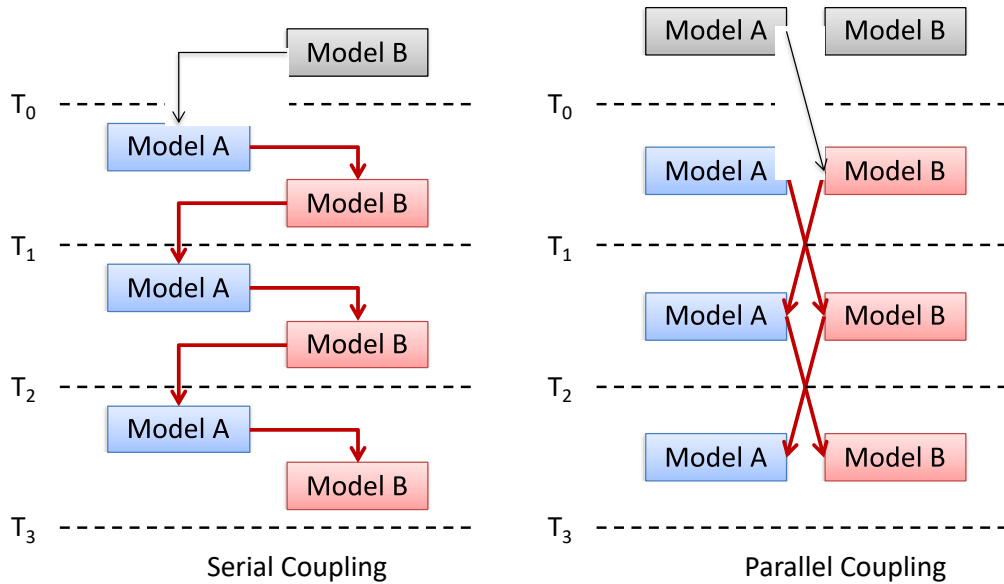


Figure 2.1: Coupling patterns (left: serial coupling, right: parallel coupling).

Chapter 3

Preparation

3.1 Grid index table

During a coupled calculation, the coupler must determine which MPI process of the target component should exchange data with each grid point. This is determined by two pieces of information: the grid point index assigned to each subdomain of the component model, and the remapping table described in the next section.

The grid point indices assigned to each subdomain are registered with the coupler through the API subroutine `rjn_def_grid`.

Indices must be positive integers. They do not need to be contiguous; non-contiguous values are allowed. However, indices must be unique across all subdomains of the component. The coupler does not check for duplicates; if duplicate indices exist, the behavior is undefined.

A single component can have multiple grids, each with an independent number of grid points and index set.

3.2 Remapping table

Raijin does not depend on any specific grid structure and can flexibly couple models whose grids do not change over time. To achieve this, the correspondence between grid indices of different models and the associated interpolation coefficients must be prepared in advance.

For example, as shown in Figure 3.1, the value at grid point $R(p)$ of the receiving model is calculated from the grid points $S(i)$ to $S(i+4)$ and the corresponding coefficients $C_s(i)$ to $C_s(i+4)$, as shown in the equation below:

$$R(p) = \sum_{n=0}^4 C_s(i+n) \cdot S(i+n) \quad (3.1)$$

The remapping table entry for $R(p)$ can be expressed as List 3.1. These values can be calculated by a remapping table generation tool such as SPRING. For more information about SPRING, see <https://hydro.iis.u-tokyo.ac.jp/~akira/page/Products/contents/SPRING/v01.03.02/>.

In Raijin, the remapping table is registered with the coupler through the API subroutine `rjn_set_mapping_table`. The sending grid indices, receiving grid indices, and interpolation coefficients are passed as arrays to this subroutine. Details of the API usage are described in a later chapter.

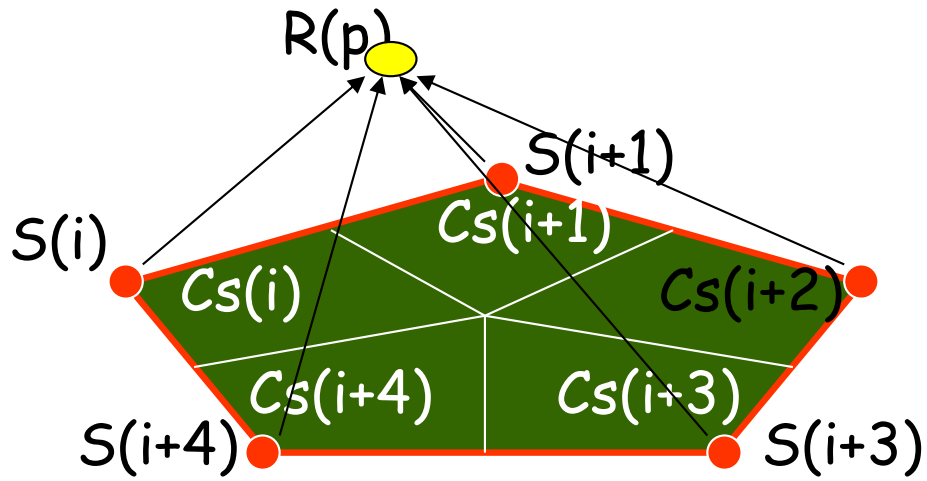


Figure 3.1: Grid point indices and interpolation coefficients.

List 3.1: Example of a remapping table entry

- | | |
|---|-------------------------|
| 1 | $R(p), S(i), Cs(i)$ |
| 2 | $R(p), S(i+1), Cs(i+1)$ |
| 3 | $R(p), S(i+2), Cs(i+2)$ |
| 4 | $R(p), S(i+3), Cs(i+3)$ |
| 5 | $R(p), S(i+4), Cs(i+4)$ |

Chapter 4

How to Use the API Subroutines

4.1 Overview of API usage

To use Raijin in a model component, include `rjn_interface.h` and link the Raijin library. Fortran compatibility is provided by `jlt_interface`; the following description uses the native Raijin `rjn_*` routine names. The Raijin API subroutines are divided into four categories: initialization, processing before time integration, processing during time integration, and finalization.

4.2 Initialization

4.2.1 Overview of initialization

A typical sequence of Raijin API calls during initialization is shown in Figure 4.1. First, initialize Raijin with `rjn_initialize`. Next, obtain the MPI parameters generated by Raijin using `rjn_get_mpi_parameter`. Then, register the grid point indices for each subdomain with `rjn_def_grid`, and notify Raijin of the end of grid definition with `rjn_end_grid_def`. After that, register the remapping tables with `rjn_set_mapping_table` and configure the exchange data with `rjn_set_data`.

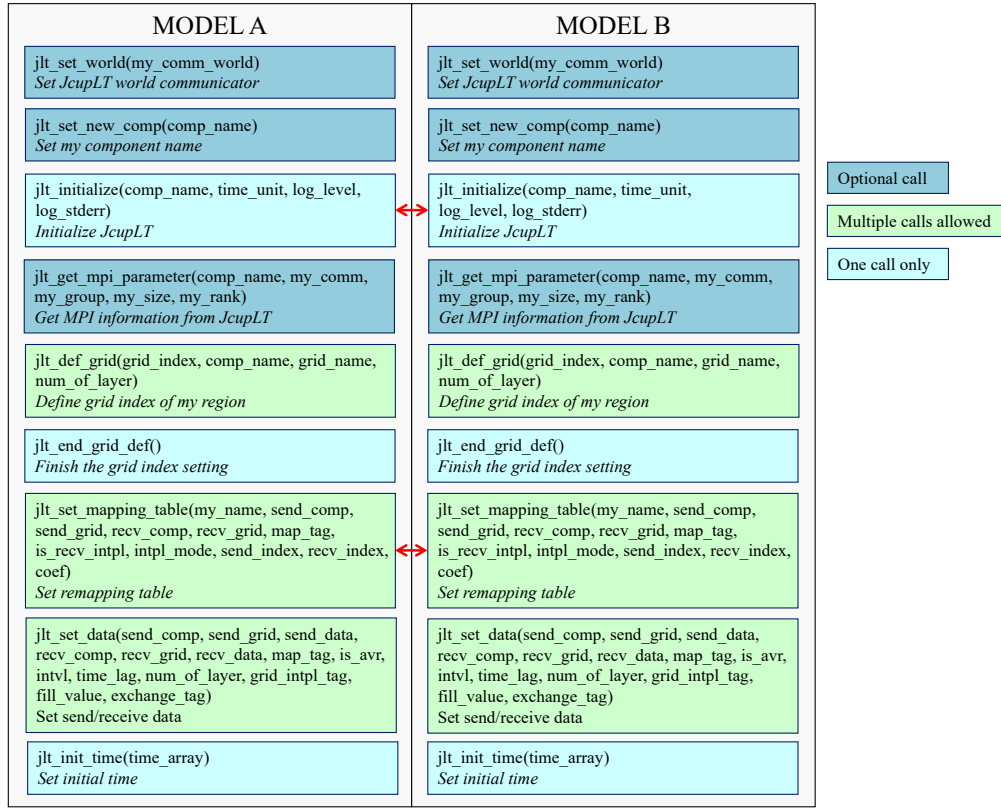


Figure 4.1: Typical sequence of Raijin API calls during initialization.

4.2.2 Pre-initialization

As shown in Figure 4.1, two subroutines, `rjn_set_world` and `rjn_set_new_comp`, can be called before the initialization routine `rjn_initialize`.

`rjn_set_world` is required only when three or more components are coupled and only a subset of them use Raijin for coupling. In such a case, a dedicated MPI communicator must be created for the Raijin-coupled components. The user must generate this communicator in advance using the MPI function `MPI_Comm_split`, and pass it to Raijin via `rjn_set_world`.

`rjn_set_new_comp` is used when a single executable contains two or more components. It registers the second and subsequent component names with Raijin. The first (i.e., primary) component name is set automatically in the initialization subroutine `rjn_initialize`.

Both of these subroutine calls can be omitted when they are not needed.

4.2.3 Initialize Raijin

Call `rjn_initialize` to initialize Raijin. The arguments are as follows: a character string representing the component name, a character string representing the time unit, an integer specifying the log output level, and a logical flag indicating whether log messages are also written to standard error output.

The time unit can be one of three values: "SEC" (seconds), "MIL" (milliseconds), or "MCR" (microseconds). This unit applies to the ΔT argument of the time-setting subroutine `rjn_set_time`, described later.

The log output level takes one of three integer values: 0 (no log output), 1 (brief log output), or 2 (detailed log output). When the log level is set to 1 or 2, log messages are written to a file named `rjn.model.name.coupling.log.PE??????`.

The last argument accepts `.true.` or `.false..` When set to `.true.` and the log level is 1 or 2, the same log messages are also written to standard error output.

4.2.4 Get MPI information

As shown in Figure 4.2, when components are coupled through Raijin, the MPI communicator for each component is generated internally by Raijin. Therefore, each component must use the Raijin-generated communicator when calling MPI routines.

The MPI parameters for each component can be obtained by calling `rjn_get_mpi_parameter`. The arguments are: the component name (input), the local communicator (output), the local MPI group ID (output), the number of local MPI processes (output), and the local MPI rank number (output).

This subroutine call is optional, but must be called if the component requires parallel computing information.

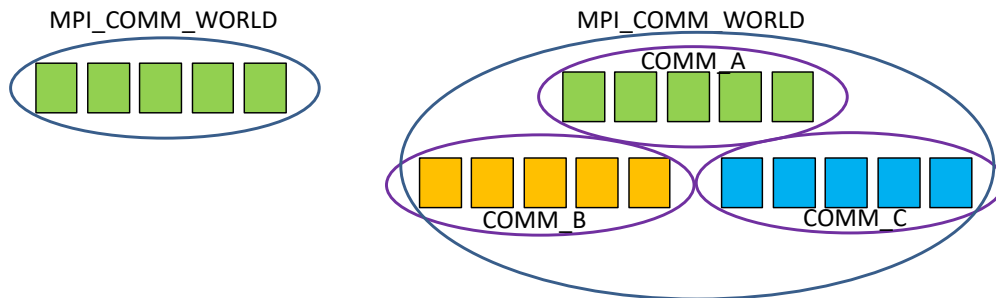


Figure 4.2: MPI communicator layout: standalone (left) vs. coupled with Raijin (right).

4.2.5 Set grid index

Call `rjn_def_grid` to register the grid point indices for each subdomain of the component. The arguments are: a one-dimensional integer array `grid_index(:)` containing the grid point indices, a character string `comp_name` specifying the component name, a character string `grid_name` specifying the grid name, and an optional integer `num_of_vgrid` specifying the number of vertical layers.

The grid point indices are always passed as a one-dimensional array, regardless of the actual grid structure. The grid size is determined from the size of the `grid_index` array.

An important consideration is whether the vertical dimension should be included in the grid index array. In many cases, such as atmosphere–ocean coupling, the exchanged data are horizontally two-dimensional. Even when a third dimension exists, it often represents a category rather than a physical vertical layer. Furthermore, even when exchanging data in a physical three-dimensional space (e.g., atmosphere–chemistry coupling), the grids may differ only in the horizontal plane, making vertical interpolation unnecessary.

In these cases, even if the exchanged data have vertical layers, provide `grid_index` as a horizontally indexed array and specify the number of vertical layers via the optional argument `num_of_vgrid` or via the `num_of_layer` argument of `rjn_set_data` (described later). The rules are summarized as follows:

- When the exchanged data are horizontally 2D, or when interpolation is performed only in the horizontal plane: provide `grid_index` as a horizontally indexed array and specify the number of vertical layers through `num_of_vgrid` or `rjn_set_data`.
- When interpolation is performed in 3D including the vertical: provide `grid_index` as an array that includes the vertical dimension.

This subroutine can be called multiple times if the component uses more than one grid system. After all grids have been registered, call `rjn_end_grid_def` to notify Raijin that the grid definition is complete.

4.2.6 Remapping table setting

Call `rjn_set_mapping_table` to register the remapping table with Raijin. The arguments are: the local component name, the sending component name, the sending grid name, the receiving component name, the receiving grid name, a table identification tag, an interpolation-side flag, an interpolation-mode string, the sending grid index array, the receiving grid index array, and the coefficient array.

The table identification tag is used to distinguish multiple remapping tables defined for the same pair of components and grids.

The interpolation-side flag takes `.true.` or `.false..` When set to `.true.`, the interpolation calculation is performed on the receiving component; when set to `.false.`, it is performed on the sending component. Note that with send-side interpolation, results may differ at the level of significant digits when the number of processes changes. With receive-side interpolation, whether binary reproducibility is maintained depends on the interpolation mode described below.

The interpolation-mode string takes one of the following values: "FAST", "SAFE", "PARALLEL", or "USER". The first three modes use the interpolation routines built into Raijin. "FAST" processes the remapping table as simply as possible; however, results may change when the number of processes changes. "SAFE" guarantees that results do not change with the number of processes. "PARALLEL" uses OpenMP parallelization for the interpolation calculation. "USER" specifies that a user-defined interpolation subroutine is used. The user-defined subroutine is registered through a Raijin API routine, described in a later section.

Note that this subroutine involves communication between the sending and receiving sides. Therefore, when registering multiple remapping tables, the order of registration must be consistent between the sending and receiving components.

4.2.7 Exchange data setting

Call `rjn_set_data` to configure the information for each exchange variable. The arguments are: the sending component name, the sending grid name, the sending data name, the receiving component name, the receiving grid name, the receiving data name, a table identification tag, a time-averaging flag, the data exchange interval, a time lag, the number of vertical layers, a data identification tag, a fill value, and a data exchange identification tag.

The table identification tag must correspond to the tag used in `rjn_set_mapping_table`.

The time-averaging flag takes `.true.` or `.false..` When set to `.true.`, the data are averaged over the exchange interval. When set to `.false.`, the data at the exchange time step are sent and received directly.

The time lag is set as a pair of values on the sending and receiving sides. The following combinations are allowed:

- (0,0): Immediate data exchange mode. In this mode, the `rjn_put_data_1d`, `rjn_put_data_25d`, `rjn_get_data_1d`, and `rjn_get_data_25d` calls (described later) perform communication synchronously on the spot. This setting is mainly used for exchanging initial values and boundary conditions before time integration.
- (-1,-1): Parallel execution mode, in which two models run concurrently and exchange data. This is the standard setting for Raijin.
- (1,-1) or (-1,1): Serial execution mode, in which two models run sequentially. The component with -1 is the preceding model.

The number of vertical layers specifies the vertical extent of the data.

The data identification tag is used to identify the variable during interpolation calculations and is referenced by user-defined interpolation subroutines.

The fill value specifies the value to be excluded from the calculation when computing time-averaged data.

The data exchange identification tag is passed to the MPI communication functions. The MPI functions use this tag to match the appropriate sender and receiver.

`rjn_set_data` must be called on both the sending and receiving sides, although the order of calls may differ between the two sides.

4.2.8 Set initial time

Call `rjn_init_time` to set the initial time. The argument is an integer array of size 6, representing year, month, day, hour, minute, and second.

4.3 Processing before time integration

4.3.1 Overview of preprocessing

Before entering the time integration loop, initial values and other necessary information can be exchanged between components. In addition, the initial time must be set, and data that will be received at the first time step must be put before entering the loop. An overview of these API calls is shown in Figure 4.3.

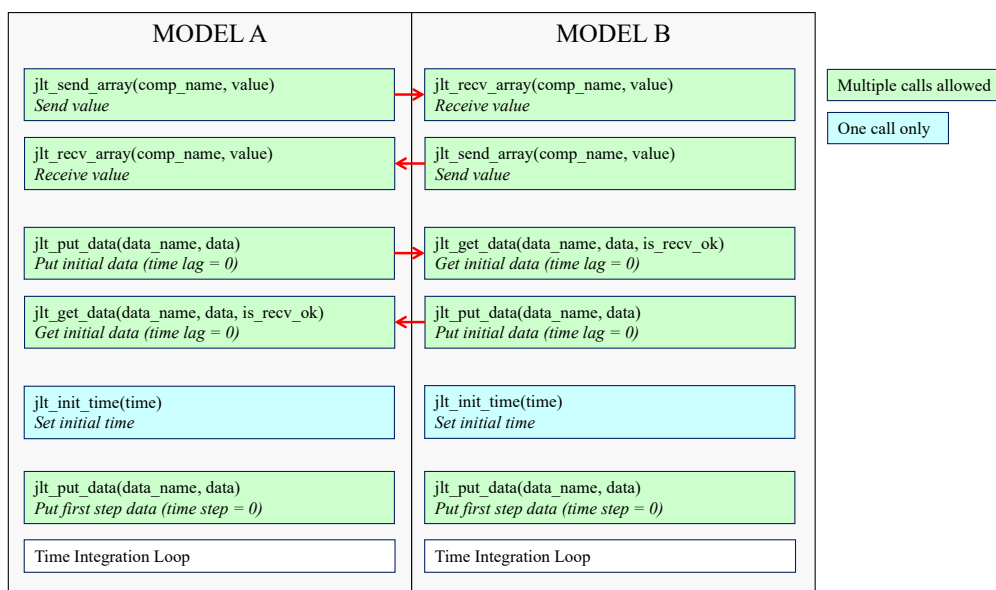


Figure 4.3: Typical sequence of Raijin API calls before the time integration loop.

4.3.2 Exchange of miscellaneous information

To exchange miscellaneous information before time integration (e.g., integration start time), use the typed routines `rjn_send_array_str`, `rjn_send_array_int`, `rjn_send_array_real`, `rjn_send_array_db1` and the corresponding `rjn_rcv_array_*` routines. The arguments are the name of the other component and the value to communicate. The value argument can be a string, integer array, real (single-precision) array, or double-precision array. The send and receive calls must correspond to each other.

4.3.3 Exchange of initial data

To exchange initial grid data (e.g., boundary conditions or static fields) before the time integration loop, use `rjn_put_data_1d` or `rjn_put_data_25d` and `rjn_get_data_1d` or `rjn_get_data_25d` for variables whose time lag is set to 0 in `rjn_set_data`. In this mode, communication is performed synchronously: `rjn_put_data_*` on the sending side and `rjn_get_data_*` on the receiving side must correspond to each other for all time-lag-0 variables.

4.3.4 Set initial time

If `rjn_init_time` was not called during the initialization phase (Section 4.2.8), it must be called here before entering the time integration loop. The argument is an integer array of size 6, representing year, month, day, hour, minute, and second.

4.3.5 Put initial values for the first time step

Before entering the time integration loop, the sending component must put the data that the receiving component will obtain at the first time step. Call `rjn_put_data_1d` for one-dimensional data, or `rjn_put_data_25d` for data with vertical layers.

4.4 Time integration

4.4.1 Overview of time integration

The Raijin API calls used in the time integration loop are shown in Figure 4.4. Three API routines are used: `rjn_set_time`, `rjn_get_data_*`, and `rjn_put_data_*`. All three routines must be called at every time step.

The data flow in Raijin differs from that of a simple send-then-receive scheme. The key points are as follows:

1. Raijin allows the interpolation calculation to be performed on either the sending or the receiving side. Which side performs the interpolation is determined by the interpolation-side flag set in `rjn_set_mapping_table`. In the example shown in Figure 4.4, Model B performs the interpolation.
2. When `rjn_put_data_1d` or `rjn_put_data_25d` is called, the data are sent immediately via non-blocking communication (`MPI_Isend`). If the current step is not a sending step, the coupler skips the send internally. Therefore, the user does not need to check the step before calling this routine.
3. Data reception (`MPI_Irecv`), completion of pending send and receive operations, and the interpolation calculation are all performed internally within `rjn_set_time`. If there is a load imbalance between components, a wait may occur in this subroutine.

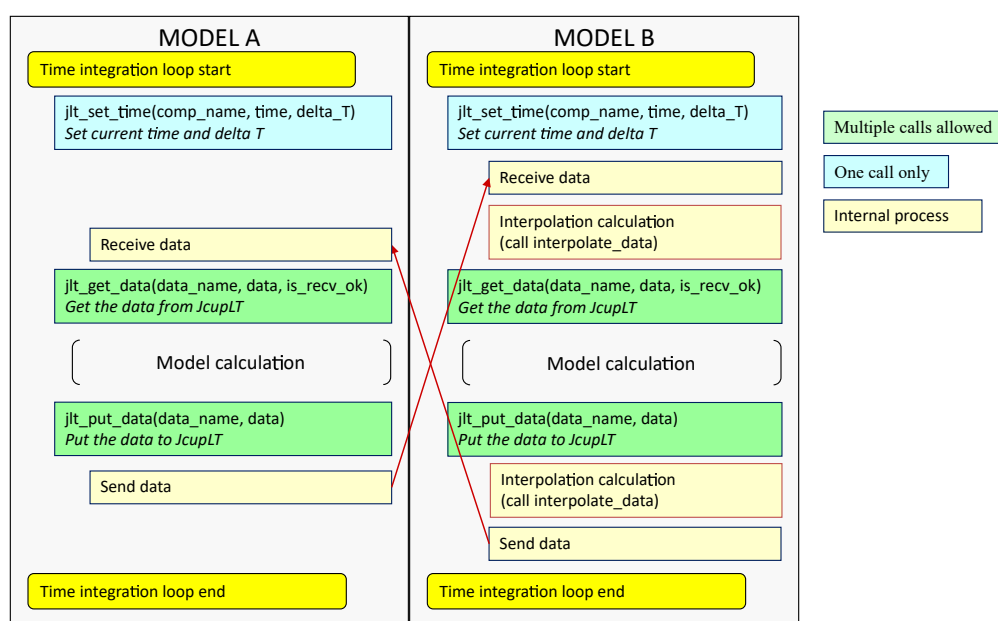


Figure 4.4: Typical sequence of Raijin API calls in the time integration loop. In this example, Model B performs the interpolation calculation.

4.4.2 Setting the current time and ΔT

Call `rjn_set_time` at each time step to pass the component name, the current time, and ΔT to Raijin. The arguments are a character string specifying the component name, an integer array of

size 6 (year, month, day, hour, minute, second), and an integer representing ΔT . The unit of ΔT is determined by the time unit specified in `rjn_initialize`.

Internally, this subroutine performs data reception, completion of pending communications, and the interpolation calculation. Because the coupler uses the accumulated value of ΔT for data exchange timing, `rjn_set_time` must be called at every step.

4.4.3 Getting data

Call `rjn_get_data_1d` or `rjn_get_data_25d` to obtain exchange data from Raijin. The argument `data_name` is the variable name, and `data` is the receiving array. The optional argument `is_recv_ok` is a logical flag that indicates whether data were actually received at this call. Because the data exchange interval does not necessarily match the model time step, use this argument to determine whether new data were received.

4.4.4 Putting data

Call `rjn_put_data_1d` or `rjn_put_data_25d` to send data to Raijin. The argument `data_name` is the variable name, and `data` is the sending data array.

When this subroutine is called at a sending step, the data are dispatched immediately via non-blocking communication. If the current step is not a sending step, the coupler skips the send internally. Therefore, the user does not need to check the step before calling this routine.

4.5 Finalization

Call `rjn_coupling_end` at the end of the coupled calculation. The first argument is an integer array representing the time at the end of the simulation. The second argument `is_call_finalize` is a flag indicating whether `MPI_Finalize` should be called internally by Raijin.

4.6 Other important routines

4.6.1 Error handling routine

The subroutine `rjn_error_ifc` outputs an error message and terminates the program. The arguments are a character string `sub_name` specifying the name of the subroutine where the error occurred, and a character string `err_str` containing the error message.

4.6.2 Component query routines

The function `rjn_get_num_of_component` returns the total number of coupled components.

The function `rjn_get_component_name` returns the component name corresponding to the given component ID number.

The function `rjn_get_comp_num_from_name` returns the component ID number corresponding to the given component name.

The functions `rjn_is_my_component_id` and `rjn_is_my_component_name` take a component ID number or component name as input and return whether the calling process belongs to that component.

4.6.3 Execution status routine

The function `rjn_is_model_running` returns whether the component specified by the argument `comp_name` is currently running.

4.6.4 MPI information routine

The function `rjn_get_myrank_global` returns the rank number of the calling process among all coupled processes.

4.6.5 Log output routine

The subroutine `rjn_log` writes a message to the Raijin log file. The arguments are a character string `sub_name` specifying the subroutine name, a character string `log_str` containing the log message, and an optional integer `log_level` specifying the log output level. The log level corresponds to the log output level set in `rjn_initialize`.

4.6.6 Calendar routines

The subroutine `rjn_inc_calendar` increments a date by a specified number of seconds. The arguments are an integer array representing the date (year, month, day, hour, minute, second) the array length, and an integer `delta_t` in seconds.

The subroutine `rjn_inc_time` increments a date by the ΔT value that was given to `rjn_set_time`. The arguments are the component name and an integer array representing the date.

4.6.7 User-defined interpolation code setting routine

The subroutine `rjn_set_user_interpolation` registers a user-defined interpolation function. The arguments `send_comp` and `send_grid` are the component name and the grid name on the sending side. The arguments `recv_comp` and `recv_grid` are the component name and the grid name on the receiving side. The argument `map_tag` is the remapping table identification tag. The argument `user_func` is a function pointer to the user-defined interpolation function. The details of this subroutine are described in a later chapter.

Chapter 5

Implementation of Coupling Code

5.1 Overview of implementation

This chapter explains how to implement coupling code, using concrete examples.

5.2 Example: toy model

The first example is a minimal C/C++ component, shown in List 5.1. The example omits domain-decomposition details and focuses on the order of Raijin API calls.

Raijin is initialized by `rjn_initialize`, and MPI information for the component is obtained by `rjn_get_mpi_parameter`. The local grid indices are registered by `rjn_def_grid`, followed by `rjn_end_grid_def`. The remapping table and exchange variables are configured by `rjn_set_mapping_table` and `rjn_set_data`. During time integration, each step calls `rjn_set_time`, receives data with `rjn_get_data_1d`, sends data with `rjn_put_data_1d`, and advances the calendar.

List 5.1: Minimal C/C++ coupling sequence

```
1 #include "rjn_interface.h"
2
3 int main(void)
4 {
5     const char* my_name = "COMP1";
6     const char* my_grid = "COMP1_GRID";
7     int time_array[6] = {2000, 1, 1, 0, 0, 0};
8     int my_comm, my_group, my_size, my_rank;
9     int local_grid_index[4] = {1, 2, 3, 4};
10    int send_index[4] = {1, 2, 3, 4};
11    int recv_index[4] = {1, 2, 3, 4};
12    double coef[4] = {1.0, 1.0, 1.0, 1.0};
13    double data[4] = {0.0, 0.0, 0.0, 0.0};
14    int is_recv_ok = 0;
15
16    rjn_initialize(my_name, "SEC", 2, 0);
17    rjn_get_mpi_parameter(my_name, &my_comm, &my_group,
18                          &my_size, &my_rank);
19
20    rjn_def_grid(local_grid_index, 4, my_name, my_grid, 1);
21    rjn_end_grid_def();
22
```

```
23  rjn_set_mapping_table(my_name,
24                        "COMP1", "COMP1_GRID",
25                        "COMP2", "COMP2_GRID",
26                        1, 0, "SAFE",
27                        send_index, recv_index, coef, 4);
28
29  rjn_set_data("COMP1", "COMP1_GRID", "varc1c2",
30              "COMP2", "COMP2_GRID", "varc1c2",
31              1, 0, 3600, -1, 1, 1, -999.0, 1);
32
33  rjn_init_time(time_array);
34  rjn_put_data_1d("varc1c2", data, 4);
35
36  for (int step = 1; step <= 24; step++) {
37      rjn_set_time(my_name, time_array, 3600);
38      rjn_get_data_1d("varc2c1", data, 4, &is_recv_ok);
39      rjn_put_data_1d("varc1c2", data, 4);
40      rjn_inc_calendar(time_array, 6, 3600);
41  }
42
43  rjn_coupling_end(time_array, 1);
44  return 0;
45 }
```

Chapter 6

Implementation of Interpolation Code

6.1 Overview

In Raijin, the basic interpolation routines are built in by default. Therefore, unless special requirements exist, users do not need to implement any interpolation code.

However, if a user needs to apply a custom interpolation algorithm, Raijin provides a mechanism for calling user-defined subroutines from within the coupling library. This chapter describes how to write and configure such user-defined interpolation subroutines.

6.2 Implementing a user-defined interpolation subroutine

The interface for a user-defined interpolation subroutine is defined as shown in List 6.1.

send_data The sending data array.

send_rows, send_cols The row and column counts of the sending data array.

recv_data The receiving data array.

recv_rows, recv_cols The row and column counts of the receiving data array.

send_index, recv_index The grid point indices on the sending and receiving sides for the interpolation calculation.

coef The interpolation coefficients.

num_of_layer The number of vertical layers, corresponding to the size of the second dimension of **send_data** and **recv_data**.

num_of_conv The size of the interpolation loop, corresponding to the array size of **send_index** and **recv_index**.

intpl_tag The data identification tag value defined for each variable in **rjn_set_data**.

The user implements the interpolation calculation using these arguments. A basic example of an interpolation subroutine is shown in List 6.2. Note that this calculation is already provided as the default implementation. How to register the subroutine with Raijin is described in the next section.

List 6.1: Interface of user-defined interpolation subroutine

```

1 typedef void (*interpolation_user_ifc)(
2     const double* send_data, int send_rows, int send_cols,
3     double* recv_data, int recv_rows, int recv_cols,
4     const int* send_index, const int* recv_index,
5     const double* coef,
6     int num_of_layer, int num_of_conv, int intpl_tag);

```

List 6.2: Example of a basic interpolation calculation

```

1 for (int k = 0; k < recv_rows * recv_cols; k++) recv_data[k] = 0.0;
2
3 for (int l = 0; l < num_of_layer; l++) {
4     for (int i = 0; i < num_of_conv; i++) {
5         int sidx = send_index[i] - 1;
6         int ridx = recv_index[i] - 1;
7         recv_data[ridx + l * recv_rows] +=
8             send_data[sidx + l * send_rows] * coef[i];
9     }
10 }

```

6.3 Registering a user-defined interpolation subroutine

A user-defined interpolation subroutine can be registered with the Raijin API routine `rjn_set_user_interpolation`.

The arguments are as follows:

`send_comp`, `send_grid` The component name and grid name on the sending side.

`recv_comp`, `recv_grid` The component name and grid name on the receiving side.

`map_tag` The mapping table identification tag.

These values must correspond to the entries specified in `rjn_set_mapping_table`.

`user_func` A procedure pointer to the user-defined interpolation subroutine, which must follow the interface described in the previous section.

6.4 Example of registering a user-defined interpolation subroutine

List 6.3 shows an example of implementing and registering a user-defined interpolation subroutine.

In this example, the function `my_interpolation` performs a nearest-neighbor interpolation instead of the default weighted-sum calculation. The function is registered by calling `rjn_set_user_interpolation` after `rjn_set_data` and before the time integration loop.

List 6.3: Example of user-defined interpolation registration

```

1 void my_interpolation(const double* send_data, int send_rows, int send_cols,
2     double* recv_data, int recv_rows, int recv_cols,
3     const int* send_index, const int* recv_index,
4     const double* coef,
5     int num_of_layer, int num_of_conv, int intpl_tag)
6 {

```

```

7   (void)send_cols;
8   (void)recv_cols;
9   (void)intpl_tag;
10
11  for (int k = 0; k < recv_rows * num_of_layer; k++) {
12      recv_data[k] = 0.0;
13  }
14
15  for (int l = 0; l < num_of_layer; l++) {
16      for (int i = 0; i < num_of_conv; i++) {
17          if (coef[i] > 0.5) {
18              int sidx = send_index[i] - 1;
19              int ridx = recv_index[i] - 1;
20              recv_data[ridx + l * recv_rows] =
21                  send_data[sidx + l * send_rows];
22          }
23      }
24  }
25 }

```

The registration is performed as follows:

List 6.4: Registering the user-defined subroutine

```

1  rjn_end_grid_def();
2
3  rjn_set_mapping_table(...);
4  rjn_set_data(...);
5
6  rjn_set_user_interpolation("COMPONENT_A", "grid_a",
7                             "COMPONENT_B", "grid_b",
8                             1, my_interpolation);
9
10 rjn_init_time(initial_time);

```

Chapter 7

API Reference

7.1 Raijin API subroutines

7.1.1 Initialization API

The initialization API subroutines are shown in Table 7.1.

`rjn_set_world` sets the MPI communicator to be used by Raijin as its global communicator. The user must generate this communicator in advance using `MPI_Comm_split` or similar MPI routines. This subroutine is required only when a subset of components use Raijin for coupling. If all components are coupled through Raijin, this call can be omitted.

`rjn_get_world` returns the MPI communicator currently used by Raijin as its global communicator.

`rjn_set_new_comp` registers an additional component name with Raijin. This subroutine is used when a single executable contains two or more components, to register the second and subsequent component names.

`rjn_initialize` initializes Raijin. The arguments are the component name, the time unit, the log output level, and a flag for standard error output. This subroutine is called only once.

`rjn_get_mpi_parameter` obtains the MPI parameters for the specified component. The component name is given as input, and the communicator, group ID, number of processes, and rank number are returned as output.

`rjn_def_grid` registers the grid used by each component. The argument `grid_index` is an integer array containing the grid point indices of the local subdomain. `comp_name` and `grid_name` specify the component and grid names. The optional argument `num_of_vgrid` specifies the number of vertical layers. This subroutine can be called multiple times if the component uses more than one grid system.

`rjn_end_grid_def` declares the end of the grid definition.

`rjn_set_mapping_table` registers a remapping table with Raijin. See the Remapping table setting section for details.

`rjn_set_data` configures the exchange data information. See the Exchange data setting section for details.

`rjn_init_time` sets the initial time of the calculation. The argument `time_array(:)` is an integer array of size 6, representing year, month, day, hour, minute, and second.

Note that these subroutines must be called in the order shown in the table.

Table 7.1: Initialization API.

Routine name	Argument type	Argument	Description
rjn_set_world	Set Raijin global communicator integer, intent(IN)	global_comm	MPI communicator
rjn_get_world	Return Raijin global communicator no argument (integer function)		
rjn_set_new_comp	Register additional component name character(len=*), intent(IN)	comp_name	component name
rjn_initialize	Initialize Raijin character(len=*), intent(IN) character(len=3), optional, intent(IN) integer, optional, intent(IN) logical, optional, intent(IN)	comp_name time_unit log_level log_stderr	component name time unit ("SEC", "MIL", "MCR") log level (0, 1, 2) stderr output flag
rjn_get_mpi_parameter	Get MPI information character(len=*), intent(IN) integer, intent(OUT) integer, intent(OUT) integer, intent(OUT) integer, intent(OUT)	comp_name my_comm my_group my_size my_rank	component name communicator group ID number of local processes local rank number
rjn_def_grid	Register grid index integer, intent(IN) character(len=*), intent(IN) character(len=*), intent(IN) integer, optional, intent(IN)	grid_index(:) comp_name grid_name num_of_vgrid	grid point indices component name grid name number of vertical layers
rjn_end_grid_def	End grid definition no argument		
rjn_set_mapping_table	Register remapping table character(len=*), intent(IN) character(len=*), intent(IN) character(len=*), intent(IN) character(len=*), intent(IN) character(len=*), intent(IN) integer, intent(IN) logical, intent(IN) character(len=*), intent(IN) integer, intent(IN) integer, intent(IN) real(kind=8), intent(IN)	my_name send_comp send_grid recv_comp recv_grid map_tag is_recv_intpl intpl_mode send_grid(:) recv_grid(:) coef(:)	local component name sending component name sending grid name receiving component name receiving grid name table identification tag interpolation side flag interpolation mode ("FAST", "SAFE", "PARALLEL", "USER") sending grid indices receiving grid indices interpolation coefficients
rjn_set_data	Configure exchange data character(len=*), intent(IN) character(len=*), intent(IN) character(len=*), intent(IN) character(len=*), intent(IN) character(len=*), intent(IN) character(len=*), intent(IN) integer, intent(IN) logical, intent(IN) integer, intent(IN) integer, intent(IN) integer, intent(IN) integer, intent(IN) real(kind=8), intent(IN)	send_comp send_grid send_data_name recv_comp recv_grid recv_data_name map_tag is_avr intvl time_lag num_of_layer grid_intpl_tag fill_value exchange_tag	sending component name sending grid name sending data name receiving component name receiving grid name receiving data name table identification tag time-averaging flag exchange interval time lag number of vertical layers data identification tag fill value exchange identification tag
rjn_init_time	Set initial time integer, intent(IN)	time_array(:)	integer array yy/mo/dd/hh/mm/ss

7.1.2 Time integration API

The API subroutines for data exchange during time integration are shown in Table 7.2.

`rjn_set_time` takes the component name, an integer array of size 6 representing the current time, and an integer representing ΔT . This subroutine must be called at the beginning of each time step. Data exchanges are performed within this subroutine based on the elapsed time calculated by accumulating ΔT .

`rjn_put_data_1d` and `rjn_put_data_25d` send data to the coupler. The arguments are the data name and the data array. `rjn_put_data_1d` is used for one-dimensional data, and `rjn_put_data_25d` is used for data with an explicit vertical-layer dimension.

`rjn_get_data_1d` and `rjn_get_data_25d` receive data from the coupler. The argument `is_recv_ok` is an integer flag that indicates whether data were actually received at this call. Because the data exchange interval does not necessarily match the model time step, use this argument to determine whether new data were received.

Table 7.2: Time integration API.

Routine name	Argument type	Argument	Description
<code>rjn_set_time</code>	Set current time and ΔT		
	character(len=*), intent(IN)	<code>comp_name</code>	component name
	integer, intent(IN)	<code>current_time(:)</code>	current time
	integer, intent(IN)	<code>delta_t</code>	ΔT
<code>rjn_put_data_1d</code>	Send one-dimensional data to coupler		
	character(len=*), intent(IN)	<code>data_name</code>	data name
	real(kind=8), intent(IN)	<code>data(:)</code>	send data
	integer, intent(IN)	<code>n_data</code>	array length
<code>rjn_put_data_25d</code>	Send data with vertical layers to coupler		
	character(len=*), intent(IN)	<code>data_name</code>	data name
	real(kind=8), intent(IN)	<code>data(:, :)</code>	send data
	integer, intent(IN)	<code>n1, n2</code>	array dimensions
<code>rjn_get_data_1d</code>	Receive one-dimensional data from coupler		
	character(len=*), intent(IN)	<code>data_name</code>	data name
	real(kind=8), intent(INOUT)	<code>data(:)</code>	receive data
	integer, intent(IN)	<code>n_data</code>	array length
<code>rjn_get_data_25d</code>	Receive data with vertical layers from coupler		
	character(len=*), intent(IN)	<code>data_name</code>	data name
	real(kind=8), intent(INOUT)	<code>data(:, :)</code>	receive data
	integer, intent(IN)	<code>n1, n2</code>	array dimensions
	integer, intent(OUT)	<code>is_recv_ok</code>	receive flag

7.1.3 Finalization API

The finalization API subroutine is shown in Table 7.3.

`rjn_coupling_end` is called at the end of the coupled calculation. The argument `time_array` is an integer array representing the time at the end of the simulation. The optional argument `is_call_finalize` is a flag indicating whether `MPI_Finalize` should be called inside `rjn_coupling_end`.

Table 7.3: Finalization API.

Routine name	Argument type	Argument	Description
<code>rjn_coupling_end</code>	Finalize coupling integer, intent(IN) logical, optional, intent(IN)	<code>time_array(:)</code> <code>is_call_finalize</code>	end time flag: call <code>MPI_Finalize</code>

7.1.4 Utility API

In addition to the initialization, time integration, and finalization subroutines described above, the following utility routines are provided.

Error handling API

Table 7.4: Error handling API.

Routine name	Argument type	Argument	Description
<code>rjn_error_ifc</code>	Output error message and terminate character(len=*), intent(IN) character(len=*), intent(IN)	<code>sub_name</code> <code>err_str</code>	subroutine name error message

Component query API

Table 7.5: Component query API.

Routine name	Argument type	Argument	Description
<code>rjn_get_num_of_component</code>	Return total number of coupled components no argument (integer function)		
<code>rjn_get_component_name</code>	Return component name from ID integer, intent(IN)	<code>comp_id</code>	component ID number
<code>rjn_get_comp_num_from_name</code>	Return component ID from name character(len=*), intent(IN)	<code>comp_name</code>	component name
<code>rjn_is_my_component_id</code>	Return whether the calling process belongs to the specified component integer, intent(IN)	<code>comp_id</code>	component ID
<code>rjn_is_my_component_name</code>	Return whether the calling process belongs to the specified component character(len=*), intent(IN)	<code>comp_name</code>	component name
<code>rjn_is_model_running</code>	Return whether the component is running character(len=*), intent(IN)	<code>comp_name</code>	component name

MPI query API

Table 7.6: MPI query API.

Routine name	Argument type	Argument	Description
<code>rjn_get_mpi_parameter</code>	Get MPI information		
	character(len=*), intent(IN)	<code>comp_name</code>	component name
	integer, intent(OUT)	<code>my_comm</code>	communicator
	integer, intent(OUT)	<code>my_group</code>	group ID
	integer, intent(OUT)	<code>my_size</code>	number of local processes
	integer, intent(OUT)	<code>my_rank</code>	local rank number
<code>rjn_get_myrank_global</code>	Return global rank number		
	no argument (integer function)		

Value send/receive API

The subroutines for sending and receiving values between two components are shown in Table 7.7.

`rjn_send_array_str`, `rjn_send_array_int`, `rjn_send_array_real`, and `rjn_send_array_dbl` send data to another component. These subroutines are meaningful only on the sending component's leader process.

`rjn_recv_array_str`, `rjn_recv_array_int`, `rjn_recv_array_real`, and `rjn_recv_array_dbl` receive data from the sending component. These subroutines must be called on all processes of the receiving component. The flag `bcast_flag` controls whether the received data are broadcast to all local processes. Send and receive calls must correspond one-to-one.

`rjn_bcast_local` broadcasts an integer array within the local component.

Table 7.7: Value send/receive API.

Routine name	Argument type	Argument	Description
rjn_send_array_str	Send a character string to another component		
	character(len=*), intent(IN)	my_comp	local component name
	character(len=*), intent(IN)	recv_comp	receiving component name
	character(len=*), intent(IN)	str	string to send
rjn_rcv_array_str	Receive a character string from another component		
	character(len=*), intent(IN)	my_comp	local component name
	character(len=*), intent(IN)	send_comp	sending component name
	character(len=*), intent(INOUT)	str	received string
	integer, intent(IN)	bcast_flag	broadcast flag
rjn_send_array_int	Send an integer array to another component		
	character(len=*), intent(IN)	my_comp	local component name
	character(len=*), intent(IN)	recv_comp	receiving component name
	integer, intent(IN)	array(:)	data to send
	integer, intent(IN)	n	array length
rjn_rcv_array_int	Receive an integer array from another component		
	character(len=*), intent(IN)	my_comp	local component name
	character(len=*), intent(IN)	send_comp	sending component name
	integer, intent(INOUT)	array(:)	received data
	integer, intent(IN)	n	array length
	integer, intent(IN)	bcast_flag	broadcast flag
rjn_send_array_real	Send a single-precision real array to another component		
	Arguments are the same as rjn_send_array_int , except for real(kind=4) data		
rjn_rcv_array_real	Receive a single-precision real array from another component		
	Arguments are the same as rjn_rcv_array_int , except for real(kind=4) data		
rjn_send_array_dbl	Send a double-precision real array to another component		
	Arguments are the same as rjn_send_array_int , except for real(kind=8) data		
rjn_rcv_array_dbl	Receive a double-precision real array from another component		
	Arguments are the same as rjn_rcv_array_int , except for real(kind=8) data		
rjn_bcast_local	Broadcast integer array within component		
	character(len=*), intent(IN)	my_comp	component name
	integer, intent(INOUT)	data(:)	integer array

Log output API

Table 7.8: Log output API.

Routine name	Argument type	Argument	Description
rjn_log	Write log message		
	character(len=*), intent(IN)	sub_name	subroutine name
	character(len=*), intent(IN)	log_str	log message
	integer, optional, intent(IN)	log_level	log output level

Calendar API

Table 7.9: Calendar API.

Routine name	Argument type	Argument	Description
rjn_inc_calendar	Increment calendar date by specified seconds		
	integer, intent(INOUT)	date(:)	calendar yy/mo/dd/hh/mm/ss
	integer, intent(IN)	delta_t	seconds to add
rjn_inc_time	Increment calendar date by ΔT from rjn_set_time		
	character(len=*), intent(IN)	comp_name	component name
	integer, intent(INOUT)	itime(:)	calendar yy/mo/dd/hh/mm/ss

User-defined interpolation API

Table 7.10: User-defined interpolation API.

Routine name	Argument type	Argument	Description
rjn_set_user_interpolation	Register user-defined interpolation function		
	character(len=*), intent(IN)	send_comp	sending component name
	character(len=*), intent(IN)	send_grid	sending grid name
	character(len=*), intent(IN)	recv_comp	receiving component name
	character(len=*), intent(IN)	recv_grid	receiving grid name
	integer, intent(IN)	map_tag	mapping table tag
	function pointer	user_func	user-defined interpolation function

List of Figures

1.1	Schematic of atmosphere–ocean interaction (from Encyclopedia of the Environment, https://www.encyclopedie-environnement.org/en/air-en/biosphere-hydrosphere-and-cryosphere-models/).	6
1.2	Schematic of data exchange between domain-decomposed models.	7
2.1	Coupling patterns (left: serial coupling, right: parallel coupling).	9
3.1	Grid point indices and interpolation coefficients.	11
4.1	Typical sequence of Raijin API calls during initialization.	13
4.2	MPI communicator layout: standalone (left) vs. coupled with Raijin (right).	14
4.3	Typical sequence of Raijin API calls before the time integration loop.	17
4.4	Typical sequence of Raijin API calls in the time integration loop. In this example, Model B performs the interpolation calculation.	18

List of Tables

7.1	Initialization API.	27
7.2	Time integration API.	28
7.3	Finalization API.	29
7.4	Error handling API.	29
7.5	Component query API.	29
7.6	MPI query API.	30
7.7	Value send/receive API.	31
7.8	Log output API.	31
7.9	Calendar API.	32
7.10	User-defined interpolation API.	32

Listings

3.1	Example of a remapping table entry	11
5.1	Minimal C/C++ coupling sequence	21
6.1	Interface of user-defined interpolation subroutine	24
6.2	Example of a basic interpolation calculation	24
6.3	Example of user-defined interpolation registration	24
6.4	Registering the user-defined subroutine	25